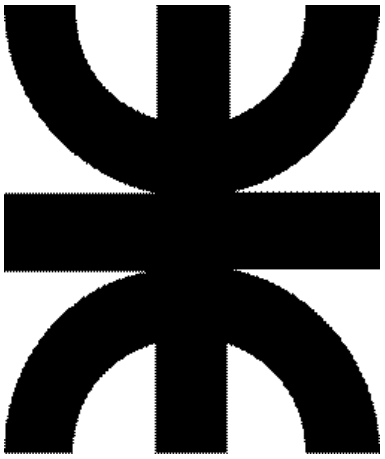


UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Resistencia

Ingeniería en Sistemas de Información

Cátedra: Desarrollo de Software

Arquitectura Orientada a Eventos (EDA) y Patrón Publicación/Suscripción (Pub/Sub)
Investigación y aplicación de conceptos de Desarrollo de Software

Tema asignado: Arquitectura orientada a eventos (EDA) y Publicación/Suscripción (Pub/Sub)

Grupo: 12

Integrantes:

Ayala Trangoni Malena Esperanza - Legajo: 27754

Di Rado Luciano - Legajo: 26957

Fankhauser Agostina - Legajo: 27311

Kowtun Andrea - Legajo: 27049

Montiel Lautaro Agustín - Legajo: 28810

Pared Chudy Nahuel - Legajo: 25696

Pozzer Maurizio - Legajo: 27420

Vignoles Francisco - Legajo: 27259

Docentes:

Jorge Eduardo Villaverde

Jose Alejandro Fernández

Resistencia, Chaco — Noviembre de 2026

1. Introducción y objetivo **4**
1.1 Objetivo general 4
1.2 Objetivos específicos 5
1.3 Organización del documento 5
2. Investigación conceptual **5**
2.1 Arquitectura orientada a eventos (EDA) 5
2.1.1 Qué es y qué problema resuelve 5
2.1.2 Componentes 6
2.1.3 Tipos de mensajes: Command, Document y Event Message 6
2.1.4 Taxonomía de Eventos: Event Notification vs. Event-Carried State Transfer (ECST) 7
Variantes Estructurales: Event Notification (Thin) vs. ECST (Fat) 7
2.1.5 Modelos de Transferencia de Estado: Snapshots vs. Deltas y Riesgos de Acoplamiento 9
2.1.6 Metodología de Clasificación de Flujos de Trabajo en Cinco Preguntas 9
2.1.7 Desafíos Operativos e Ingeniería de Replicación en ECST 9
2.1.8 Coordinación de Flujos de Trabajo Distribuidos: Orquestación vs. Coreografía 10
2.1.8.1 Orquestación Basada en Comandos y Control Centralizado 10
2.1.8.2 Coreografía Descentralizada Basada en Eventos 10
2.1.8.3 Marco de Decisión Arquitectónica y Modelos Híbridos 11
2.1.9 Beneficios y limitaciones de EDA 12
2.2 Publicación/Suscripción (Pub/Sub) 12
2.2.1 Qué es y qué problema resuelve 13
2.2.2 Punto a punto vs. Pub/Sub 14
2.2.3 Características 14
2.2.4 Garantías de entrega, orden y duplicados 14
2.2.5 Gestión de Fallos de Entrega: El Patrón Dead Letter Channel (DLQ) 15
2.2.6 Beneficios y limitaciones 15
2.3 Cómo se relacionan los dos conceptos 15
2.4 Tecnologías de implementación 16
2.4.1 Apache Kafka 16
2.4.2 RabbitMQ 17
2.4.3 Cuál elegir 17
3. Aplicación al Trabajo Práctico Integrador **17**
4. Criterios de utilización y análisis crítico **21**
4.1 Cuándo conviene EDA y Pub/Sub, y cuándo no 21
4.2 Alternativas: REST síncrono, colas punto a punto 22
4.3 Análisis por factor: complejidad, costo, rendimiento, mantenibilidad 22
4.4 Aplicado al TP: en qué partes sí y en cuáles no 23
5. Implementación práctica integrada **23**
5.1 Objetivo y alcance 23
5.2 Arquitectura implementada 24
5.3 Modelo de eventos 25
5.4 Topología del broker 26
5.5 Microservicios 27

5.6 Flujos principales 28
5.6.1 Creación de un pedido 28
5.6.2 Cancelación de un pedido 29
5.6.3 Reintento duplicado e idempotencia 29
5.6.4 Pedido inválido y DLQ 29
5.6.5 Caída y recuperación de un consumidor 29
5.7 Panel de observación 29
5.8 Decisiones de diseño y justificación 30
5.9 Limitaciones y mejoras 30
5.10 Evidencias de funcionamiento 31
5.11 Cierre de la sección 36
6. Conclusiones **36**
7. Bibliografía **38**

1. Introducción y objetivo

Los sistemas distribuidos están formados por componentes autónomos que se ejecutan en procesos distintos y cooperan intercambiando mensajes a través de la red. En este tipo de sistemas, el modo en que se comunican los servicios condiciona directamente su disponibilidad, su escalabilidad y su mantenibilidad. Cuando la integración se basa en invocaciones síncronas directas, cada servicio depende de que sus interlocutores estén disponibles y respondan a tiempo en el mismo momento. Así, una falla o un aumento de latencia en un componente se propaga a los que lo invocan. Además, el servicio que origina una operación tiene que conocer de forma explícita a todos los que necesitan enterarse de ella, de modo que incorporar un nuevo interesado obliga a modificar el emisor. Microsoft (s. f., Event-Driven Architecture Style)

La plataforma distribuida de movilidad urbana desarrollada en el Trabajo Práctico Integrador (TPI) de la asignatura presenta este problema de manera concreta. A lo largo del ciclo de vida de un viaje ocurren hechos de negocio relevantes, como la solicitud de un viaje, la asignación de un conductor o la finalización del recorrido. Estos hechos interesan a varios procesos independientes entre sí: la asignación de conductores, el envío de notificaciones y la generación de estadísticas, entre otros. Resolver esta distribución mediante llamadas directas acoplaría a esos procesos con el servicio que origina cada hecho, tanto en el tiempo como en la estructura del sistema.

Este trabajo investiga dos temas complementarios que abordan este problema: la **Arquitectura Orientada a Eventos** (*Event-Driven Architecture*, EDA) y el patrón de **Publicación/Suscripción** (*Publish/Subscribe*, Pub/Sub). La EDA es un estilo arquitectónico en el que los servicios se comunican mediante la emisión de eventos de negocio y la reacción a ellos, en lugar de invocarse directamente. El patrón Pub/Sub es el mecanismo de comunicación que permite que un mismo evento, publicado una sola vez en un intermediario (*broker*), sea entregado a múltiples consumidores independientes sin que el publicador los conozca. Ambos temas se estudian en conjunto porque son complementarios: la EDA define qué se comunica y bajo qué principios de diseño, mientras que Pub/Sub define cómo se distribuye esa información entre los interesados. La investigación se orienta por dos preguntas guía:

- ¿Cómo distribuir un evento de negocio a varios consumidores independientes sin acoplarlos al servicio que lo publica?
- ¿Cómo diseñar un sistema cuyos servicios se comunican mediante eventos de negocio en lugar de llamadas directas, y cómo gestionar el desacoplamiento y la consistencia eventual que ello implica?

1.1 Objetivo general

Analizar cómo la Arquitectura Orientada a Eventos, implementada mediante el patrón Publicación/Suscripción, permite desacoplar los componentes de la plataforma de movilidad urbana, evaluando a la vez los costos que introduce en términos de consistencia, complejidad y operación.

1.2 Objetivos específicos

1. Explicar los fundamentos de la EDA y del patrón Pub/Sub: los elementos que los componen (evento, productor, broker y consumidor), sus características, beneficios y limitaciones.
2. Distinguir la EDA como estilo arquitectónico del patrón Pub/Sub como mecanismo de comunicación, y diferenciar Pub/Sub de la comunicación punto a punto.
3. Identificar los eventos de negocio del ciclo de vida de un viaje, tomando como fuente las transiciones de estado gestionadas por el módulo M6, y determinar qué módulos de la plataforma los publican y cuáles los consumen.
4. Establecer criterios para decidir cuándo conviene aplicar estos conceptos y cuándo no, en función de la complejidad, el costo, el rendimiento y la mantenibilidad, comparándolos con alternativas como la comunicación síncrona (REST).
5. Implementar y demostrar un escenario funcional que integre ambos temas y evidencie el desacoplamiento entre consumidores, el procesamiento diferido y la recuperación ante la detención de un consumidor.

1.3 Organización del documento

La **sección 2** desarrolla el marco conceptual de la EDA y del patrón Pub/Sub: tipos de mensajes y de eventos, modelos de coordinación (orquestación y coreografía), garantías de entrega y limitaciones.

La **sección 3** vincula estos conceptos con el TPI: identifica los eventos del ciclo de vida de un viaje, los módulos productores y consumidores y el rol del módulo M6.

La **sección 4** presenta los criterios de utilización y un análisis crítico frente a las alternativas.

La **sección 5** describe la implementación práctica integrando los dos temas en un simulacro de ecommerce.

Por último, la **sección 6** expone las conclusiones y la **sección 7** reúne la bibliografía consultada.

2. Investigación conceptual

2.1 Arquitectura orientada a eventos (EDA)

2.1.1 Qué es y qué problema resuelve

La Arquitectura Orientada a Eventos (EDA, por sus siglas en inglés) constituye un paradigma de diseño en sistemas distribuidos que establece el desacoplamiento temporal y espacial como principio operacional primario. En las arquitecturas compuestas por microservicios, la comunicación síncrona mediante llamadas a procedimientos remotos vincula el ciclo de vida de los componentes en una dependencia de ejecución simultánea, donde la indisponibilidad o latencia de una entidad degrada linealmente la disponibilidad del sistema entero. La adopción de EDA mitiga esta vulnerabilidad al interponer un canal de transporte asíncrono donde las

interacciones no requieren la coexistencia temporal de los participantes ni el conocimiento explícito de su topología de red. Microsoft (s. f., Event-Driven Architecture Style)

2.1.2 Componentes

Una arquitectura orientada a eventos se apoya en cuatro elementos básicos:

Evento: es el registro de algo que ya pasó en el sistema y que le importa a otros (por ejemplo, "se creó un pedido"). Como ya ocurrió, no se modifica: solo se informa.

Productor (emisor): es el componente donde ocurre el hecho y que lo comunica generando el evento. No sabe quién va a recibirlo ni qué hará con él.

Canal o broker de mensajería: es la infraestructura intermedia (por ejemplo, Kafka o RabbitMQ) que recibe los eventos del productor, los conserva hasta que se procesen y los entrega a quienes corresponda. Es lo que permite que productor y consumidor no se conozcan ni necesiten estar disponibles al mismo tiempo.

Consumidor (receptor): es el componente que recibe el evento y reacciona con su propia lógica, de forma independiente. Un mismo evento puede ser consumido por varios consumidores, y se pueden sumar nuevos sin tocar el productor. Microsoft (s. f., Event-Driven Architecture Style)

La comunicación es asíncrona: el productor publica el evento y sigue con su trabajo sin esperar una respuesta.

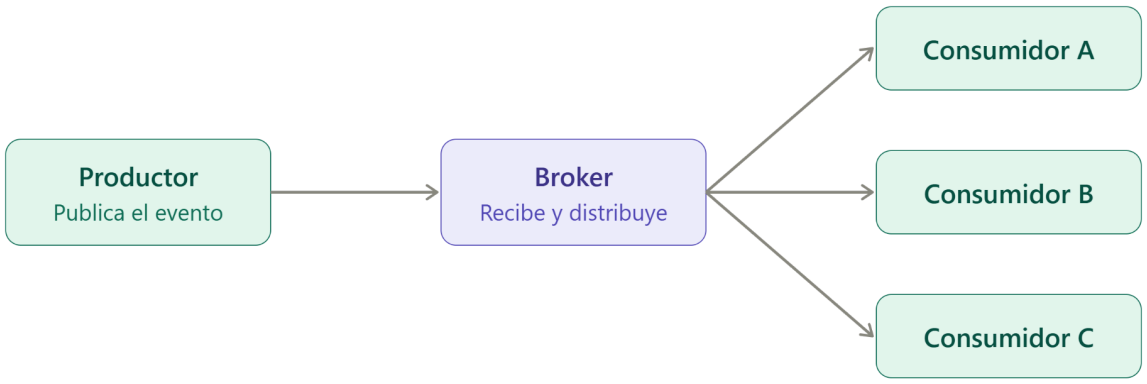


Figura 2.1 Componentes de una arquitectura orientada a eventos.

2.1.3 Tipos de mensajes: Command, Document y Event Message

La especificación de Patrones de Integración Empresarial (EIP) establece una categorización rigurosa de las estructuras de transporte en función de su intención semántica y su grado de acoplamiento operacional:

- **Command Message (Mensaje de Comando)**: Codifica la intención explícita del emisor de invocar un procedimiento o alterar el estado en un receptor específico. Su

semántica exige la ejecución de una acción y asume que el emisor conoce las capacidades del destinatario, generando un acoplamiento funcional fuerte.

- **Document Message (Mensaje de Documento)**: Transporta una estructura de datos neutra a través de los límites del sistema. Su propósito primario es la transferencia de información sintáctica; no instruye al receptor sobre qué operación ejecutar ni notifica un cambio de estado en el dominio.
- **Event Message (Mensaje de Evento)**: Notifica la ocurrencia de un hecho inmutable ocurrido en el pasado dentro del dominio del emisor. El emisor no emite suposiciones sobre si existirá o no un receptor, eliminando la direccionalidad de la invocación. Hohpe y Woolf (2003)

Tipo de Mensaje	Intención Semántica	Direccionalidad/Acoplamiento	Efecto en el Receptor
Mensaje de Comando	Exigir la ejecución de una acción o procedimiento específico.	Dirigido a un receptor único conocido. Acoplamiento funcional fuerte.	Ejecuta la instrucción solicitada o retorna un fallo.
Mensaje de Documento	Transferir estructuras de datos para su almacenamiento o procesamiento.	Dirigido a un consumidor específico o compartido. Acoplamiento sintáctico.	Procesa o almacena los datos recibidos según su propia lógica.
Mensaje de Evento	Notificar un hecho inmutable e histórico ocurrido en el dominio.	Difusión pública o sin suscripción (*broadcast*). Alto desacoplamiento espacial y temporal.	Reacciona de manera autónoma o ignora la notificación.

Tabla 2.1 Tipos de mensajes: Command, Document y Event Message

2.1.4 Taxonomía de Eventos: Event Notification vs. Event-Carried State Transfer (ECST)

La selección de la granularidad y la amplitud de los datos (*payload*) contenidos en un evento define de forma crítica las características operacionales de un sistema distribuido. Esta decisión de diseño altera directamente el perfil de disponibilidad del sistema, la latencia en el procesamiento, el ancho de banda consumido en la red y el acoplamiento entre fronteras de dominio.

Variantes Estructurales: Event Notification (Thin) vs. ECST (Fat)

La dicotomía entre eventos delgados (*thin*) y eventos gordos (*fat*) contrapone dos filosofías de transferencia de información:

- **Event Notification (Thin)**: Transmite el contrato mínimo necesario para señalar que ha ocurrido un cambio relevante. Contiene habitualmente identificadores primarios y metadatos del hecho. Al recibir la notificación, el consumidor puede requerir un callback si no dispone del estado local mediante API (ej. HTTP GET) hacia el servicio origen para recuperar los detalles requeridos.
- **Event-Carried State Transfer (Fat / ECST)**: Incluye dentro de la estructura del evento todo el estado necesario para que los consumidores ejecuten sus procesos de negocio

locales de manera autónoma, casi eliminando la necesidad de consultas de retorno hacia el servicio emisor. Fowler (2017)

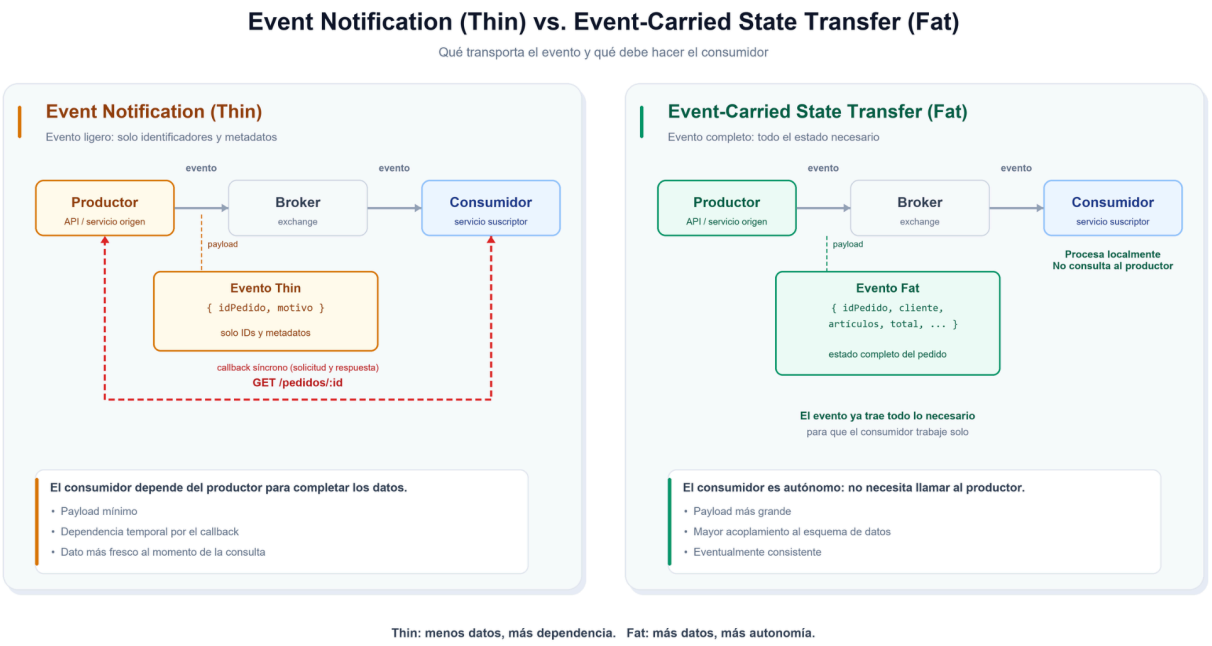


Figura 2.2. Event Notification (Thin) vs. Event-Carried State Transfer (Fat).

Dimensión	Event Notification (Thin)	Event-Carried State Transfer (Fat)
Tamaño de Mensaje	Mínimo; típicamente restringido a IDs, tipo de evento y marcas de tiempo.	Elevado; contiene estructuras complejas de datos y colecciones asociadas.
Disponibilidad del Origen	Alta requerida; el origen debe estar en línea para responder a los *callbacks*.	Baja requerida; el origen puede colapsar sin paralizar la ejecución del consumidor.
Carga de Red	Baja en el canal de mensajería; potencialmente alta por picos de tráfico HTTP hacia el origen.	Alta en el broker de mensajería; nula en llamadas síncronas de lectura.
Frescura del Dato	Máxima al momento de ejecutar el *callback* directo al sistema de verdad.	Eventualmente consistente; sujeta al retraso de procesamiento del consumidor.
Complejidad del Contrato	Muy baja y estable; sufre pocas variaciones en el tiempo.	Alta; cambios en el esquema impactan a múltiples consumidores expuestos.

Replicación de Almacenamiento	Inexistente; el dato reside únicamente en el sistema dueño (*source of truth*).	Alta; cada consumidor persiste un duplicado local especializado de los datos.

Tabla 2.2 Variantes Estructurales: Event Notification (Thin) vs. ECST (Fat)

2.1.5 Modelos de Transferencia de Estado: Snapshots vs. Deltas y Riesgos de Acoplamiento

En la implementación del patrón ECST, la estructuración de los datos del estado se clasifica en dos enfoques:

1. **Snapshot**: Transporta la representación completa del estado resultante tras el cambio. Permite al consumidor sobrescribir directamente su proyección local de forma determinista.
2. **Delta**: Transporta únicamente los campos específicos modificados en comparación con el estado inmediatamente anterior, reduciendo el tamaño del payload a expensas de exigir lógica de fusión (*merge*) en el receptor.

El antipatrón *"Fat Event"* surge cuando los diseñadores agregan campos heterogéneos y no relacionados a un evento de forma *ad-hoc* para satisfacer necesidades puntuales de múltiples consumidores distintos. Esto contamina el contrato público del servicio origen, exponiendo detalles internos de la implementación de la base de datos y acoplando la evolución del productor a las demandas individuales de los receptores. Fowler (2017)

2.1.6 Metodología de Clasificación de Flujos de Trabajo en Cinco Preguntas

Para discernir de forma metódica entre la adopción de *Event Notification* o *Event-Carried State Transfer*, se debe evaluar el flujo a través de las siguientes cinco cuestiones analíticas:

1. **¿Qué decisión de negocio tomará el consumidor?** Determina la frontera mínima de datos requerida estrictamente para ejecutar la lógica del receptor sin exponer entidades completas por defecto.
2. **¿Tolerará una búsqueda síncrona (*callback*)?** Evalúa si el caso de uso soporta la latencia agregada de red y si la tasa de llamadas puede saturar la disponibilidad del servicio emisor (*thundering herd: pico masivo de llamadas*).
3. **¿Qué estado es legítimo replicar?** Filtra datos sensibles, credenciales de autenticación o banderas internas que violen normativas de privacidad o fronteras de seguridad si se transmiten ampliamente.
4. **¿Actúa sobre un hecho actual o coordina un paso de trabajo?** Distingue si la intención es consumir un dato estático para consulta o desencadenar una transición en un flujo transaccional.
5. **¿Cuál es la consecuencia del estado replicado desactualizado?** Define la tolerancia del negocio a operar bajo consistencia eventual; si el proceso exige validez estricta e instantánea, la réplica desactualizada resulta inviable.

2.1.7 Desafíos Operativos e Ingeniería de Replicación en ECST

La eliminación del acoplamiento en tiempo de ejecución vía ECST introduce responsabilidades operativas complejas en la gestión de datos distribuida:

- **Consistencia Eventual**: Acepta que la proyección local del consumidor mantendrá un desfase temporal respecto al emisor. La lógica de negocio debe redactarse asumiendo ventanas de desactualización.
- **Reglas de Ordenamiento y Versionado**: Dado que el transporte asíncrono no garantiza entrega ordenada entre particiones o reconexiones, el productor debe adjuntar un número de secuencia o versión del estado (`version`). El consumidor aplicará el cambio únicamente si se cumple la condición estricta `versión_entrante > versión_almacenada`, descartando mutaciones obsoletas o desordenadas.
- **Idempotencia en Procesamiento**: Ante entregas duplicadas provocadas por reintentos de la red, la aplicación de un *Snapshot* a la misma versión debe producir un resultado idéntico sin corromper la proyección local.
- **Estrategias de Inicialización (*Bootstrapping*)**: Un consumidor nuevo requiere construir su base de datos local desde cero. El diseño de la infraestructura subyacente impone restricciones operativas fundamentales en este proceso. (ver 2.4) Microsoft (s. f., Event-Driven Architecture Style)

2.1.8 Coordinación de Flujos de Trabajo Distribuidos: Orquestación vs. Coreografía

El diseño de flujos de negocio que abarcan múltiples microservicios expone la tensión entre el control explícito centralizado y la autonomía descentralizada de los componentes.

2.1.8.1 Orquestación Basada en Comandos y Control Centralizado

La orquestación establece un modelo de comunicación basado en comandos (*command-driven*), donde un servicio centralizado (el Orquestador o *Process Manager*) asume la responsabilidad de dirigir de forma explícita la secuencia de ejecución del flujo de trabajo. El orquestador envía instrucciones puntuales a los microservicios participantes y aguarda sus respuestas para determinar la siguiente transición de estado.

Ventajas

- **Visibilidad Centralizada**: Facilidad para monitorear, auditar y rastrear el estado global del flujo desde un único componente.
- **Manejo de Excepciones**: Coordinación simplificada de transacciones compensatorias en caso de fallos parciales.
- **Control explícito del flujo**: Garantía de ejecución explícita de pasos condicionales complejos.

Desventajas

- **Acoplamiento Estrecho**: El orquestador conoce la existencia y las capacidades de los servicios colaboradores.
- **Punto Único de Fallo**: La indisponibilidad del orquestador paraliza el flujo de trabajo completo.
- **Lógica Concentrada**: Riesgo de degradar los microservicios periféricos a meros contenedores anémicos de datos. Richardson (s. f., Saga)

2.1.8.2 Coreografía Descentralizada Basada en Eventos

La coreografía adopta un modelo descentralizado y conducido por eventos (*event-driven*). No existe un controlador central. Cada microservicio reacciona de manera autónoma ante la

publicación de eventos de dominio por parte de otros componentes, ejecuta su lógica interna y, opcionalmente, emite nuevos eventos al canal para que terceros reaccionen.

Ventajas

- **Alto Desacoplamiento**: Los servicios ignoran la existencia, cantidad o topología de los consumidores que reaccionan a sus eventos.
- **Escalabilidad y Resiliencia**: La falla de un servicio intermedio no detiene la ingesta ni la emisión de hechos por parte de otros componentes.
- **Evolución Autónoma**: Facilidad para agregar nuevos receptores sin modificar el código de los productores existentes.

Desventajas

- **Complejidad Operativa**: Extrema dificultad para visualizar el estado global de un proceso de negocio.
- **Desafío en Trazabilidad (*Tracing*)**: La depuración y correlación de incidentes distribuibles exige infraestructura avanzada de telemetría.
- **Riesgo de Bucles Cíclicos**: Eventos emitidos en cadena pueden desencadenar ciclos de ejecución infinitos no detectados en compilación. Richardson (s. f., Saga)

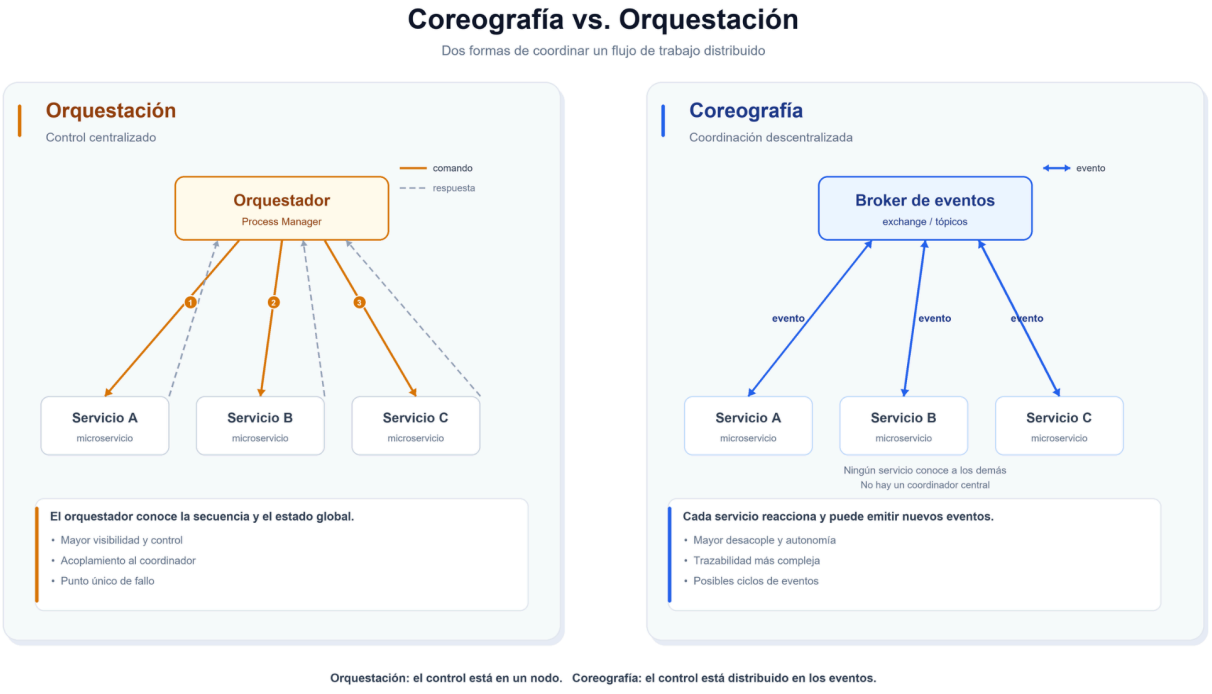


Figura 2.3. Coreografía vs. orquestación.

2.1.8.3 Marco de Decisión Arquitectónica y Modelos Híbridos

Para la selección del enfoque adecuado se deben ponderar las siguientes reglas de decisión:

- **Orquestación es superior cuando**: El flujo contiene bifurcaciones condicionales complejas, requiere confirmaciones intermedias obligatorias, exige un modelo de

transacciones distribuidas con compensación (Saga) , o el dominio requiere trazabilidad centralizada para cumplimiento regulatorio.
- **Coreografía es superior cuando**: Los pasos progresan en una única dirección, las interacciones son altamente asíncronas y de gran volumen, cada componente puede operar únicamente con la información contenida en el mensaje de entrada, y la flexibilidad para añadir consumidores supera el costo del monitoreo.

En arquitecturas distribuidas enterprise, la solución óptima suele adoptar un **modelo híbrido**. Dentro de los límites de un mismo Contexto Delimitado (*Bounded Context:* contexto delimitado), las interacciones se coordinan mediante un Orquestador interno para mantener la consistencia del flujo. Para la comunicación entre diferentes *Bounded Contexts*, la integración se realiza mediante Coreografía basada en eventos, preservando la autonomía de las fronteras del dominio.

El establecimiento de fronteras de dominio desacopladas mediante coreografía u orquestación exige garantizar que los contratos de datos transmitidos a través de los eventos permanezcan inmutables y compatibles en el tiempo mediante la integración de un registro de esquemas centralizado. Richardson (s. f., Saga)

2.1.9 Beneficios y limitaciones de EDA

Los beneficios y límites de EDA dependen de cómo se diseña el sistema. Los del mecanismo de transporte (broker, duplicados, orden, fallos de entrega) se tratan en 2.2.6. (Microsoft, s. f.)

Beneficios

Hechos explícitos: cada evento deja registrado qué ocurrió en el dominio, lo que facilita la auditoría y permite que otros servicios reaccionen.

Extensibilidad: una función nueva se agrega escuchando eventos que ya existen, sin modificar los servicios actuales (ver 2.1.8.2).

Servicios autónomos: con eventos completos (ECST), los consumidores pueden trabajar sin consultar al productor (ver 2.1.4).

Limitaciones

Consistencia eventual: los servicios no ven el mismo estado al mismo tiempo (ver 2.1.7).

Flujo implícito: no hay un punto que muestre el proceso completo, por lo que seguirlo exige identificadores de correlación y herramientas de seguimiento (ver 2.1.8.2).

Contratos compartidos: cambiar un evento puede afectar a varios consumidores a la vez (ver Tabla 2.2).

Los criterios de cuándo conviene o no aplicar EDA se analizan en la sección 4.

2.2 Publicación/Suscripción (Pub/Sub)

2.2.1 Qué es y qué problema resuelve

Publicación/Suscripción es un patrón de mensajería en el que quien envía un mensaje (el publicador) no se lo manda a un destinatario concreto, sino que lo publica en un canal, llamado tópico. Los componentes interesados (los suscriptores) se suscriben a ese tópico y reciben una copia de cada mensaje publicado.

El problema que resuelve aparece cuando la comunicación es directa. Si un servicio tiene que avisar a otros tres que algo ocurrió, necesita conocer a cada uno, llamarlos por separado y modificar su propio código cada vez que se suma un cuarto. Además, si alguno de ellos está caído, la comunicación falla. Con Pub/Sub, el publicador envía el mensaje una sola vez y desconoce cuántos suscriptores hay, quiénes son o si están disponibles. Hohpe y Woolf (2003, Publish-Subscribe Channel)

En la terminología de Pub/Sub se habla de “tópicos” y “suscripciones”. En RabbitMQ, esos conceptos se materializan de otra forma: el exchange de tipo topic recibe cada mensaje con una routing key y lo copia a cada cola que tenga un binding cuya binding key coincida. No existe un objeto llamado “tópico”: el tema lógico se representa mediante esa routing key y las reglas de vinculación. RabbitMQ (s. f., AMQP 0-9-1 Model Explained)

La Figura 2.3 muestra el mecanismo:

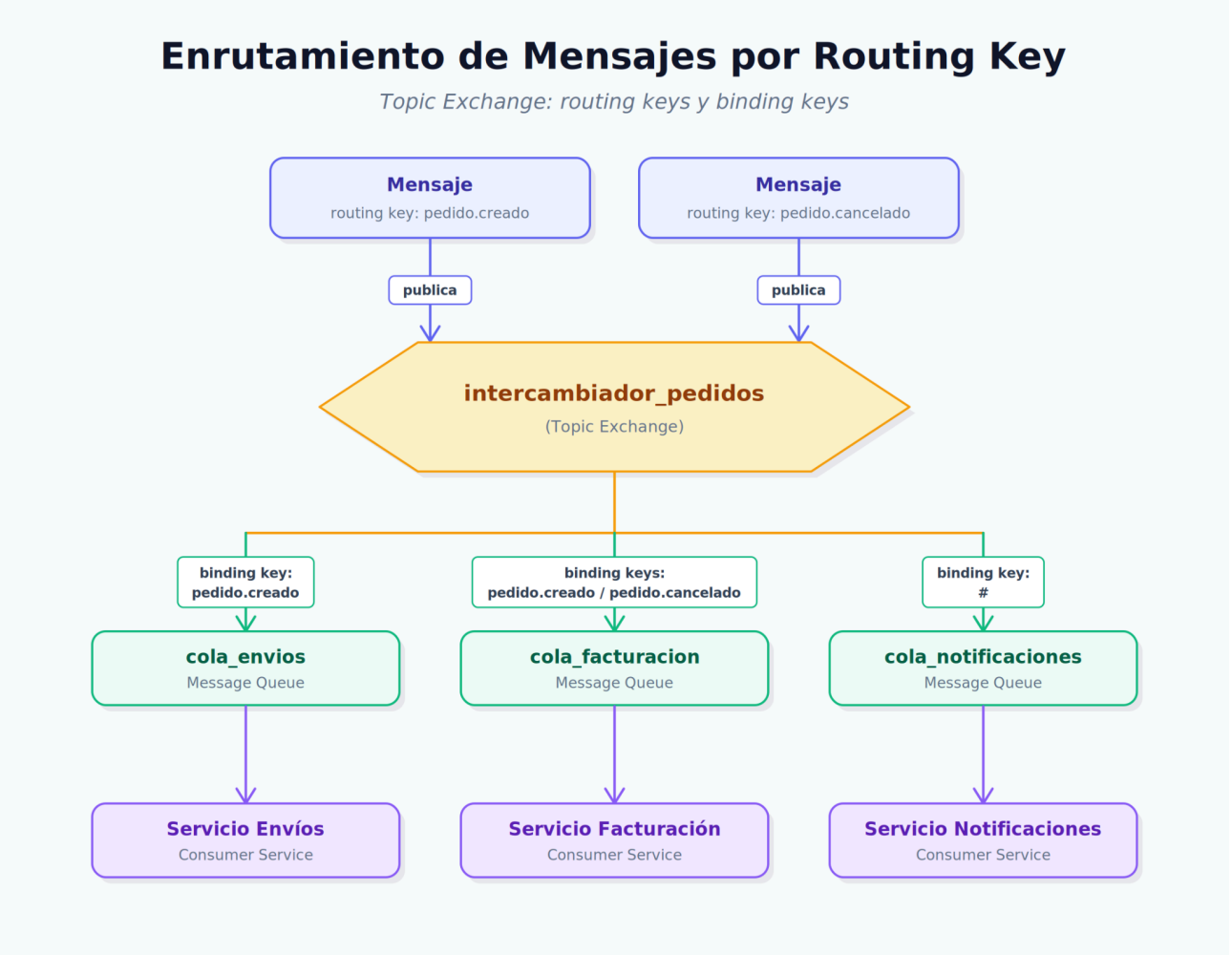


Figura 2.3 Enrutamiento selectivo en un exchange de tipo topic. El mensaje se copia a cada cola cuya binding key coincide con la routing key; el comodín # coincide con todos los tópicos.

2.2.2 Punto a punto vs. Pub/Sub

La topología del transporte de datos distingue operacionalmente dos modelos fundamentales: el canal punto a punto (*Point-to-Point Channel*) y el canal de publicación/suscripción (*Publish-Subscribe Channel*). En un canal punto a punto, un mensaje garantizado es consumido por un único destino; si múltiples consumidores compiten por el canal, la infraestructura asegura la distribución unitaria del mensaje, manteniendo la semántica de procesamiento exclusivo. Por el contrario, el patrón Publicación/Suscripción (*Publish-Subscribe*) distribuye cada mensaje transmitido a todos los suscriptores activos en el canal. El emisor publica un *Event Message* únicamente caracterizado por el contexto de su propio dominio. El middleware de mensajería asume la responsabilidad de replicar la entrega a múltiples consumidores concurrentes. Esta abstracción elimina la dependencia de red directa: el productor no gestiona direcciones IP, nombres de host ni esquemas de descubrimiento para los receptores, aislando al emisor de los fallos, la escala o las adiciones dinámicas de consumidores en el ecosistema distribuido. Hohpe y Woolf (2003, Competing Consumers)

2.2.3 Características

- **Desacople espacial:** publicador y suscriptores no se conocen ni necesitan saber dónde está el otro.
- **Desacople temporal:** no hace falta que estén conectados al mismo tiempo. Si el broker conserva el mensaje, un suscriptor que estuvo caído lo recibe cuando vuelve.
- **Desacople de ejecución:** el publicador no espera a que los suscriptores terminen; sigue trabajando después de publicar.
- **Tópicos y suscripciones:** los mensajes se organizan por tema y cada suscriptor elige a cuáles se suscribe. Algunos brokers permiten además filtrar por contenido. Eugster et al. (2003)

Pub/Sub es uno de los mecanismos que hace posible la arquitectura orientada a eventos: EDA define qué se comunica (eventos) y Pub/Sub define cómo se reparte.

2.2.4 Garantías de entrega, orden y duplicados

Que un mensaje llegue una sola vez y en orden parece obvio, pero en un sistema distribuido es difícil de garantizar. Hay tres niveles de garantía de entrega:

- **A lo sumo una vez:** el mensaje llega una vez o no llega. Nunca se duplica, pero puede perderse.
- **Al menos una vez:** el mensaje llega seguro, pero puede llegar repetido. Es lo más habitual.
- **Exactamente una vez:** llega una sola vez. Es costoso de lograr, y suele conseguirse combinando “al menos una vez” con consumidores que toleran repetidos.

Por eso los consumidores deben ser **idempotentes**: procesar el mismo mensaje dos veces tiene que dar el mismo resultado que procesarlo una. Por ejemplo, si llega dos veces “viaje finalizado”, el segundo no debe generar un cobro duplicado.

Con el **orden** pasa algo parecido. Los brokers suelen garantizarlo solo dentro de una misma partición o cola, no entre varias. Si el orden importa, se publican los mensajes relacionados con la misma clave para que caigan juntos, y se agrega un número de versión: el consumidor aplica el cambio solo si la versión recibida es mayor que la que ya tiene, y descarta los mensajes viejos o desordenados. Microsoft (s. f., Publisher-Subscriber)

2.2.5 Gestión de Fallos de Entrega: El Patrón Dead Letter Channel (DLQ)

¿Qué pasa cuando un mensaje se corrompe o el consumidor se rompe al intentar procesarlo? No podemos permitir que un mensaje defectuoso sature la cola principal para siempre.

Para eso existe la **Cola de Mensajes Muertos (Dead Letter Queue - DLQ)**: si un mensaje falla de manera repetida tras agotar los reintentos permitidos (por ejemplo, controlando el contador *Approximate ReceiveCount* en Amazon SQS (Simple Queue Service) o configurando las tolerancias en Kafka Connect), la infraestructura lo aparta automáticamente hacia la DLQ. Esto aísla el error, evita bucles infinitos de fallas y deja el registro listo para que los desarrolladores lo auditen sin frenar el flujo principal del sistema. Hohpe y Woolf (2003, Dead Letter Channel)

2.2.6 Beneficios y limitaciones

Beneficios

- Desacople entre quien publica y quienes consumen.
- Se pueden agregar suscriptores nuevos sin modificar al publicador.
- Cada suscriptor escala y falla de forma independiente.
- Si un suscriptor se cae, no frena a los demás ni al publicador.

Limitaciones

- El broker pasa a ser una pieza crítica: si no está bien configurado o replicado, un fallo afecta a todo el sistema.
- Aparecen duplicados y problemas de orden que hay que manejar en cada consumidor.
- Los datos son eventualmente consistentes, ya que cada suscriptor los recibe con un pequeño retraso.
- Es más difícil depurar: no hay una única cadena de llamadas, y cuesta seguir qué hizo cada suscriptor con un mensaje.
- No sirve cuando el emisor necesita una respuesta inmediata (petición y respuesta) o cuando solo hay un único consumidor.
- Hay costo extra de operación: instalar, monitorear y mantener el broker. Microsoft (s. f., Publisher-Subscriber)

2.3 Cómo se relacionan los dos conceptos

EDA y Pub/Sub no son sinónimos ni alternativas: operan en **niveles distintos** y se complementan.

- **EDA (Arquitectura Orientada a Eventos)** es un **estilo arquitectónico**. Define *qué* se comunica y *por qué*: el sistema se organiza alrededor de hechos del dominio (eventos),

los servicios se desacoplan y la coordinación es, típicamente, por coreografía. Es una decisión de diseño del sistema. Fowler (2017)

- **Pub/Sub (Publicación/Suscripción)** es un **mecanismo de transporte**. Define *cómo* se distribuyen esos eventos: un productor publica un mensaje una sola vez, un broker lo recibe y lo entrega a los suscriptores interesados. Es una decisión de infraestructura de mensajería. Hohpe y Woolf (2003), Eugster et al. (2003)

Dimensión	**EDA**	**Pub/Sub**
Naturaleza	Estilo arquitectónico	Patrón de mensajería
Pregunta que responde	¿Qué hechos se comunican y cómo se coordina el dominio?	¿Cómo viajan los mensajes y quién los recibe?
Elementos	Eventos, productores, consumidores, coreografía, consistencia eventual	Broker, exchange, colas, bindings, suscripciones
En el proyecto	PedidoCreado, PedidoCancelado, eventos derivados, idempotencia, ECST/Thin	RabbitMQ, intercambiador_pedidos, cola_envios, #, DLQ

Tabla 2.3 Relacion entre EDA y Pub/Sub

La relación puede resumirse así: **EDA es el estilo; Pub/Sub es el medio de transporte.** La primera decide que el e-commerce avise "se creó un pedido" en lugar de ordenar "facturá este pedido"; el segundo se encarga de que ese aviso llegue a las colas correctas sin que el productor conozca a los consumidores.

Es importante notar que **pueden existir por separado**. Se puede usar Pub/Sub sin EDA, por ejemplo para enviar notificaciones o distribuir tareas; y se puede implementar EDA con otros mecanismos de transporte, como un log de eventos o webhooks. Esto confirma que Pub/Sub es un **medio**, no un fin arquitectónico.

En la solución desarrollada, EDA se manifiesta en la coreografía y en la taxonomía de eventos; Pub/Sub se manifiesta en el exchange topic, los bindings y las colas. La arquitectura define el comportamiento esperado; el broker garantiza que los eventos lleguen a quienes deben reaccionar.

2.4 Tecnologías de implementación

Cuando EDA se combina con Pub/Sub necesitan un broker que reciba, guarde y reparta los mensajes. Dos de los más usados son Apache Kafka y RabbitMQ. Sirven para lo mismo, pero están pensados de forma distinta y eso condiciona cómo se diseña el sistema.

2.4.1 Apache Kafka

Kafka guarda los mensajes en un log persistente: un registro ordenado al que se van agregando eventos y que se conserva en disco según una política de retención configurable. Que un consumidor lea un mensaje no lo borra, así que otros consumidores pueden leer el mismo mensaje, y uno nuevo puede empezar desde el principio del historial.

Kafka también permite tópicos compactados (cleanup.policy=compact), que conservan solo el último valor de cada clave. Gracias a esto, un consumidor nuevo puede reconstruir su estado

local leyendo el tópico: ya sea el historial completo o la última versión válida de cada dato, sin pedirle nada al servicio que publicó. Documentación de Apache Kafka (falta en la bibliografía)

2.4.2 RabbitMQ

RabbitMQ está pensado sobre todo para el enrutamiento y la entrega de mensajes: los mensajes llegan a un intercambiador (exchange), que los distribuye a colas según reglas, y se eliminan de la cola cuando el consumidor confirma que los procesó. Por eso, en su funcionamiento clásico no guarda un historial de mensajes ya consumidos.

Esto tiene una consecuencia práctica: cuando se suma un consumidor nuevo, no puede reconstruir su estado releyendo lo anterior. Hay que resolverlo con otros mecanismos, como leer un snapshot guardado aparte (por ejemplo, en un almacenamiento de objetos) o hacer una carga masiva de datos mediante la API del servicio de origen (un *backfill: carga masiva de datos históricos*). RabbitMQ (s. f., AMQP 0-9-1 Model Explained)

2.4.3 Cuál elegir

Elegir depende de lo que necesite el sistema. Si importa conservar el historial de eventos o que consumidores nuevos puedan reconstruir su estado, encaja Kafka. Si lo principal es repartir mensajes con reglas de enrutamiento flexibles y un broker más simple de instalar y mantener, encaja RabbitMQ. En proyectos chicos o académicos, también puede alcanzar con alternativas más livianas.

3. Aplicación al Trabajo Práctico Integrador

3.1 Breve descripción de la plataforma y del módulo M6

La plataforma de movilidad urbana está concebida como un sistema distribuido compuesto por múltiples microservicios autónomos que colaboran entre sí para gestionar todo el flujo de un viaje compartido. Dentro de esta arquitectura, el módulo M6 (“Viajes y Ciclo de Vida”) es el componente encargado de administrar la máquina de estados del viaje. M6 garantiza que las transiciones avancen únicamente bajo reglas de negocio válidas (RF-6.1). Controla la concurrencia (RNF-08) mediante actualizaciones condicionales al estado esperado y versionado con ETag / If-Match. Asegura la idempotencia (RNF-09) mediante el requestId único en la creación del viaje y la cabecera Idempotency-Key en las acciones. Registra además cada transición en una tabla de auditoría de solo inserción (TripEventHistory, RF-6.8), escrita en la misma transacción que el cambio de estado.

Conviene distinguir dos conceptos que comparten el nombre “evento”. TripEventHistory es un registro de auditoría interna de M6. Los eventos de dominio que se estudian en este trabajo son los mensajes que M6 publica hacia el resto de la plataforma a través del broker. Por su responsabilidad, M6 es la fuente de verdad (source of truth) del estado de cada viaje y el productor de los eventos que rigen su ciclo de vida, definidos en el contrato de la API (RNF-03).

M6 mantiene además dos interacciones síncronas, analizadas en la sección 3.6: recibe de M5 la creación del viaje mediante POST /trips y consulta a M8 para validar el código QR al iniciar el viaje.

3.2 Qué hechos del ciclo de vida de un viaje son eventos

Cada cambio de estado confirmado en la máquina de estados de M6 es un hecho del pasado que interesa a otros dominios de la plataforma, y se publica como evento asíncrono:

TripCreated: M5 solicita crear el viaje (POST /trips, idempotente por requestId) y M6 lo registra en estado PENDING_ARRIVAL.

DriverArrived: el conductor informa su llegada al punto de retiro (DRIVER_ARRIVED).

TripStarted: el conductor inicia el viaje, una vez que M8 validó el código QR de un solo uso (IN_PROGRESS). Si M8 no responde, M6 devuelve 503, el estado no cambia y el evento no se publica.

TripCompleted: el conductor finaliza el viaje (COMPLETED). Incluye la distancia informada por el conductor y la duración exacta calculada por M6.

TripCancelled: el viaje se cancela (CANCELLED). Pueden cancelarlo el cliente o el conductor antes de iniciar el viaje, o el operador en cualquier estado no terminal. Quién canceló se informa en el campo cancelledBy.

Todos comparten un sobre común (EventEnvelope) con eventId (clave de deduplicación), eventType, eventVersion, occurredAt, tripId, payload y, de forma opcional, correlationId para el seguimiento entre módulos (RNF-14).

3.3 Qué módulos publican y cuáles se suscriben

Productor: M6 (Viajes y Ciclo de Vida) publica todos los eventos del ciclo de vida en el exchange m6.events (tipo topic, durable), con una routing key por evento: trip.created, trip.arrived, trip.started, trip.completed y trip.cancelled.

Suscriptores. Cada módulo consume desde su propia cola (m7.trips, m8.trips, etc.), enlazada solo a las routing keys que le interesan. Los mensajes son persistentes y cada consumidor los deduplica por eventId.

Evento	Routing key	Consumidores	Qué hacen con él
TripCreated	trip.created	M8	M8 genera el QR de un solo uso (RF-8.3).
DriverArrived	trip.arrived	M8	Avisa al cliente que el conductor llegó (RF-8.1).

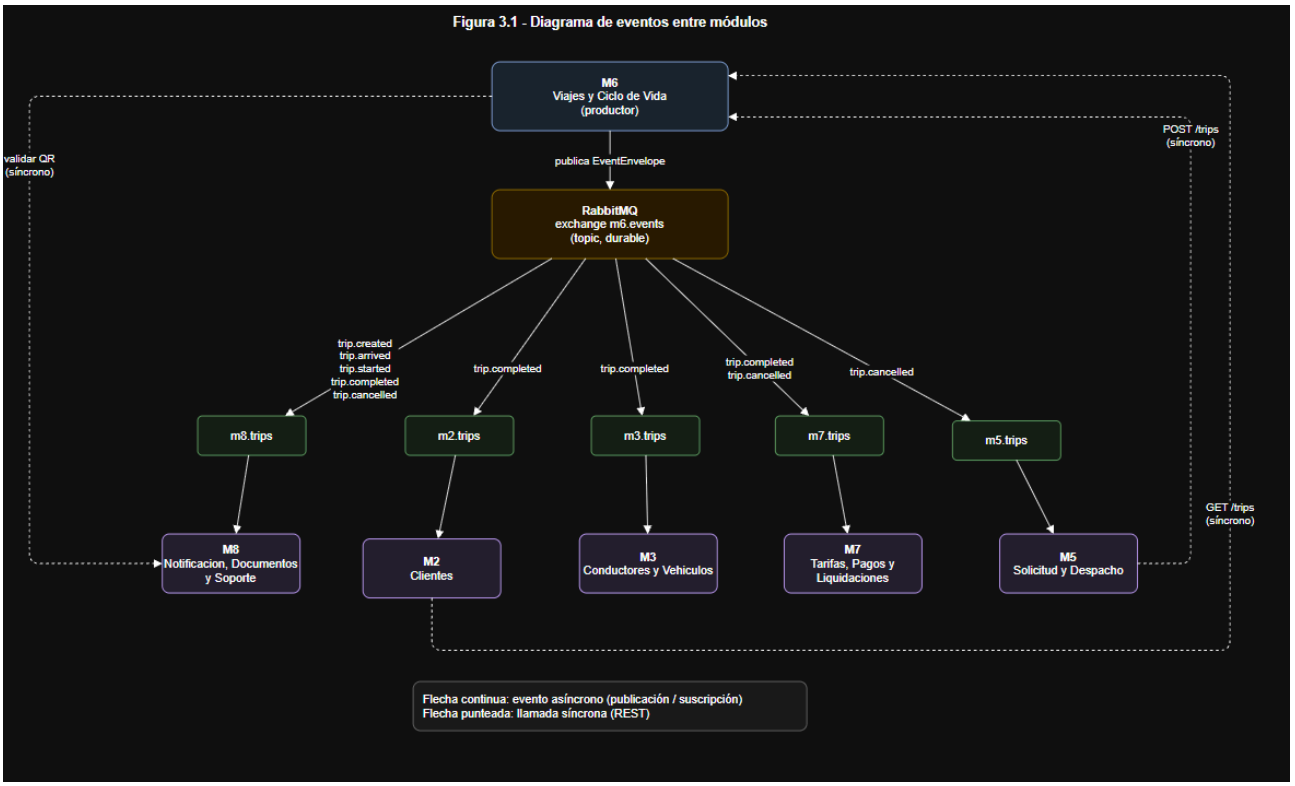
TripStarted	trip.started	M8	Avisa el inicio del viaje (RF-8.1).
TripCompleted	trip.completed	M7, M8, M2, M3	M7 cobra y liquida (RF-7.3 / RF-7.8). M8 emite el comprobante (RF-8.4). M2 y M3 habilitan las calificaciones (RF-2.5 / RF-3.7).
TripCancelled	trip.cancelled	M7, M8, M5	M7 calcula el cargo de cancelación (RF-7.4). M8 avisa a la otra parte. M5 re-despacha solo si requiresRedispatch es true.

El historial de viajes de M2 no se obtiene por eventos: M2 lo consulta por REST (GET /trips). M5 nunca recibe dos pedidos de asignación por el mismo viaje: con una cancelación del operador, requiresRedispatch es siempre false, y si M5 re-despacha debe usar un requestId nuevo.

Garantía de publicación. M6 publica cada evento después de confirmar el cambio de estado en la base de datos. Para que ningún evento se pierda si el proceso falla entre ambos pasos, se aplica el patrón transactional outbox (Richardson, s. f.): el evento se guarda en la misma transacción que el cambio de estado y un proceso aparte lo publica en el broker.

3.4 Diagrama de eventos entre módulos

La Figura 3.1 muestra la topología de comunicación. M6 publica el EventEnvelope en el exchange m6.events, y el broker copia cada mensaje a la cola de cada módulo suscriptor según las routing keys de la sección 3.3: M8, M2, M3, M7 y M5. Aparte, con línea punteada, se representan las tres interacciones síncronas: M5 → M6 (creación del viaje), M6 → M8 (validación del QR) y M2 → M6 (obtener historial).



3.5 Qué eventos serían thin y cuáles fat, y por qué

Eventos livianos (thin): DriverArrived y TripStarted. Su payload solo lleva los identificadores clientId y driverId (más el tripId del sobre). Son señales de aviso, que M8 usa para notificar al pasajero (RF-8.1). Ningún consumidor necesita consultar a M6 para procesarlas, así que no hay callback.

Eventos completos (fat, ECST):

- TripCreated es el más completo. Transporta requestId, clientId, driverId, vehicleId, vehicleType, reservationId y las coordenadas y direcciones de retiro y destino. Así M8 genera el QR y.
- TripCompleted transporta clientId, driverId, vehicleId, vehicleType, startedAt, completedAt y el objeto closure. Dentro de closure van distanceKm (informada por el conductor) y durationSeconds (duración exacta calculada por M6, el valor que M7 usa para tarifar). Se incluye también durationMinutes, solo para mostrar, ya redondeado para que ningún consumidor redondee por su cuenta: dos redondeos distintos producirían cobros distintos.
- TripCancelled transporta requestId, clientId, driverId, cancelledBy, reason, detail, previousState, secondsSinceCreation y requiresRedispatch. M7 calcula el cargo con secondsSinceCreation y reason. M5 decide con requiresRedispatch si busca otro conductor.

Con eventos completos, los consumidores no saturan a M6 con consultas de lectura (el problema del thundering herd de 2.1.6). El costo es un contrato más grande y compartido (Tabla 2.2): por eso los consumidores ignoran los campos que no conocen, y eventVersion solo sube con cambios incompatibles.

3.6 Dónde se aplicaría coreografía y dónde orquestación

Coreografía. Es el modelo de las salidas de M6. Cuando M6 publica un evento como TripCompleted, no existe un controlador central: M7, M8, M2 y M3 reaccionan de forma autónoma a través del broker, y se pueden sumar consumidores sin modificar M6. Dentro de M6 no hay orquestación: su máquina de estados es lógica interna del dominio, no un coordinador de otros servicios.

Interacciones síncronas. Tres interacciones requieren respuesta inmediata y por eso no son eventos:

M5 → M6: POST /trips, un comando que crea el viaje.

M6 → M8: validación del QR en POST /trips/{id}/start, que condiciona la transición de estado.

M2 → M6: GET /trips, consulta del historial de viajes del cliente.

El desacoplamiento es amplio en las salidas de M6, pero no total.

Orquestación. Según los criterios de la sección 2.1.8.3, la orquestación se justifica cuando el flujo tiene bifurcaciones condicionales complejas o requiere transacciones distribuidas con compensación (Saga), como un cobro que falla y obliga a revertir pasos anteriores. Ninguno de los flujos de M6 lo requiere: sus transiciones son lineales y la cancelación no necesita deshacer pasos anteriores, porque el cargo lo calcula M7 a partir del evento.

4. Criterios de utilización y análisis crítico

4.1 Cuándo conviene EDA y Pub/Sub, y cuándo no

La adopción de una Arquitectura Orientada a Eventos basada en el patrón Pub/Sub no constituye una solución universal, sino una decisión arquitectónica que debe ponderarse según las características del dominio:

Cuándo conviene:

- Cuando un único hecho de negocio (por ejemplo, la finalización de un viaje) interesa de manera simultánea a múltiples dominios independientes entre sí (facturación, calificaciones, auditoría, pasarelas de pago).
- Cuando se requiere un **desacoplamiento temporal y espacial**, permitiendo que los consumidores procesen los mensajes a su propio ritmo o permanezcan temporalmente fuera de línea sin interrumpir la operación del emisor.
- Ante escenarios de alto volumen de transacciones donde es imperativo absorber picos de tráfico sin colapsar los servicios periféricos (amortiguamiento mediante el broker).

Cuándo no conviene:

- Cuando el caso de uso exige una semántica estricta de **petición-respuesta inmediata** (bloqueante), donde el emisor necesita el resultado síncrono del receptor para continuar su propia ejecución transaccional.
- Cuando existe un único consumidor y la lógica del flujo es estrictamente secuencial y acoplada.
- En sistemas de baja complejidad donde la infraestructura adicional de un broker de mensajería (Kafka, RabbitMQ) introduce un costo operativo injustificado frente a una

base de datos compartida o llamadas directas simples. Microsoft (s. f., Event-Driven Architecture Style)

4.2 Alternativas: REST síncrono, colas punto a punto

Para dimensionar las ventajas de EDA y Pub/Sub, es necesario contrastarlas frente a sus alternativas arquitectónicas principales:

REST síncrono (HTTP/JSON):

- Funcionamiento: Invocación directa y bloqueante donde el cliente espera la respuesta HTTP del servidor.
- Evaluación: Es óptimo para operaciones de consulta directa de lectura (ej. `GET /trips/{id}`) donde se requiere consistencia inmediata. Sin embargo, aplicado a notificaciones entre microservicios genera un acoplamiento temporal estricto: si el servicio receptor cae o experimenta alta latencia, arrastra al emisor a un fallo en cascada.

Colas Punto a Punto (Point-to-Point):

- Funcionamiento: Canales de mensajería donde un mensaje publicado es consumido y procesado por **un único destinatario** (típicamente bajo el patrón de consumidores competitivos).
- Evaluación: Excelente para distribuir tareas pesadas de procesamiento asíncrono entre múltiples *workers* de un mismo servicio (balanceo de carga de trabajo). No obstante, resulta insuficiente para resolver escenarios de integración donde múltiples dominios empresariales distintos necesitan enterarse de la misma ocurrencia de negocio de forma independiente (necesidad de difusión o *broadcast* propia de Pub/Sub).

4.3 Análisis por factor: complejidad, costo, rendimiento, mantenibilidad

La elección de este estilo arquitectónico impacta de manera directa en cuatro dimensiones clave de la ingeniería de software:

1. **Complejidad:** Elevada. Obliga a diseñar mecanismos robustos para lidiar con la naturaleza de los sistemas distribuidos: gestión de idempotencia ante reintentos de red, control de ordenamiento y versiones, manejo de consistencia eventual y estrategias de aislamiento ante fallos mediante Colas de Mensajes Muertos (DLQ).
2. **Costo:** Mayor costo de infraestructura, despliegue y operación. Exige aprovisionar, configurar, escalar y monitorear componentes de broker dedicados, además de requerir herramientas de observabilidad avanzada (como identificadores de correlación - `correlationId`) para rastrear el flujo de una petición a través de múltiples servicios.
3. **Rendimiento:** Altamente favorable en términos de escalabilidad y resiliencia horizontal. El productor no bloquea sus hilos esperando respuestas de terceros, lo que optimiza el tiempo de respuesta de las APIs principales. La penalización se restringe a una mínima latencia de red inherente al tiempo de propagación del mensaje en el broker.

4. **Mantenibilidad:** Presenta una dualidad. Por un lado, facilita enormemente la evolución del sistema a largo plazo, ya que permite incorporar nuevos módulos suscriptores sin modificar una sola línea de código en el servicio productor (M6). Por otro lado, dificulta el diagnóstico y la depuración local (*debugging*), debido a la ausencia de una pila de llamadas síncronas centralizada.

5. Implementación práctica integrada

5.1 Objetivo y alcance

El escenario elegido para la demostración que integra los dos conceptos investigados es un e-commerce simplificado en el que un pedido dispara, de forma asíncrona, la reserva de stock, la facturación, la programación de un envío y el registro de notificaciones.

El objetivo no es construir un sistema listo para producción, sino una **demo verificable** que permita observar en tiempo real las propiedades centrales de EDA y Pub/Sub: desacople espacial, desacople temporal, enrutamiento selectivo, consistencia eventual, idempotencia y aislamiento de fallos.

La solución se compone de **cinco microservicios** y un **broker de mensajería** (RabbitMQ), todos contenerizados con Docker. La Tabla 5.1 resume los componentes.

Componente	Rol	Tecnología
api-pedidos	Productor de eventos y API REST; además sirve el panel web	Node.js, Express, amqplib
servicio-inventario	Consumidor y productor de eventos de inventario	Node.js, amqplib
servicio-facturación	Consumidor y productor de eventos de facturación	Node.js, amqplib
servicio-envios	Consumidor y productor de eventos de logística	Node.js, amqplib
servicio-notificaciones	Consumidor global y auditor de la cadena de eventos	Node.js, amqplib
rabbitmq	Broker de mensajería y enrutamiento	RabbitMQ 3 (imagen 3-management-alpine)

Tabla 5.1. Componentes de la implementación.

El código se organiza en un repositorio único (ecommerce-eda-demo) con una carpeta por servicio. Los archivos principales son:

Archivo	Función
docker-compose.yml	Define los seis contenedores, la red interna y las variables de entorno
api-pedidos/src/index.js	API REST, publicación de eventos y proyección de pedidos
api-pedidos/src/public/index.html	Panel web de observación (cuatro pestañas)

servicio-*/src/index.js	Consumidores: topología, procesamiento, idempotencia y DLQ
demo.ps1 / demo.bat	Guión de demostración por consola con cinco escenarios
README.md	Documentación, arquitectura y pasos de ejecución

Aclaracion: El código completo de la demo se encuentra en el repositorio público: https://github.com/Malenatrangoni/ecommerce-eda-demo.git . Las instrucciones de ejecución están en el README del repositorio.

5.2 Arquitectura implementada

La arquitectura sigue un modelo de **coreografía**: no existe un coordinador central que ordene a los servicios qué hacer. El productor publica un hecho del dominio y cada consumidor reacciona según su propia lógica, pudiendo a su vez publicar nuevos eventos.

El flujo general es el siguiente:

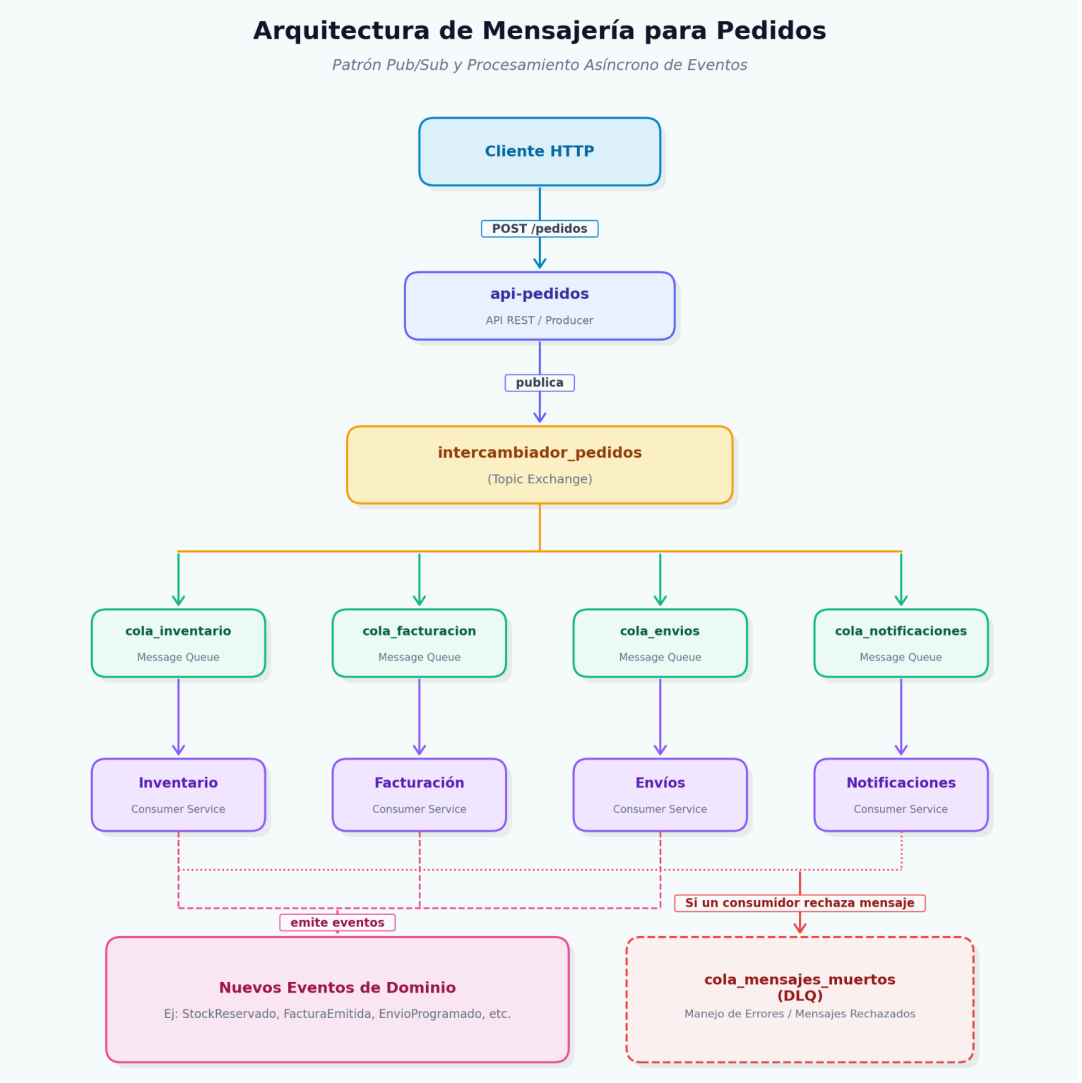


Figura 5.1. Arquitectura general de la solución.

La comunicación entre servicios no es directa: **ningún microservicio invoca por HTTP a otro para completar el flujo principal**. La API de pedidos solo conoce el exchange y la routing key; desconoce cuántos consumidores existen, dónde están y si están disponibles. Esa es la base del **desacople espacial**.

5.3 Modelo de eventos

Todos los mensajes comparten un mismo sobre o envoltorio, denominado EventEnvelope. Su función es separar los **metadatos de trazabilidad** de los **datos del dominio**.

Campo	Descripción
idEvento	Identificador único del evento; se usa para idempotencia y deduplicación
tipoEvento	Nombre del hecho ocurrido (PedidoCreado, StockReservado, etc.)
productor	Servicio que emitió el evento
topico	Routing key utilizada para publicar (pedido.creado, inventario.reservado, etc.)
patronDatos	Indica si el evento es ECST (completo) o Event Notification (ligero)
fechaHora	Marca temporal ISO 8601
idPedido	Identificador del pedido al que pertenece el evento
idCorrelacion	Identificador estable para reconstruir toda la cadena del pedido
version	Versión del agregado o del evento, usada para descartar mensajes obsoletos
eventoCausanteId	(En eventos derivados) idEvento del evento que provocó la reacción
datos	Carga útil específica del dominio

Tabla 5.2. Campos del sobre de evento.

La implementación utiliza dos variantes de eventos:

- **ECST (*Event-Carried State Transfer*) / evento completo (fat):** PedidoCreado transporta el estado completo del pedido (cliente, artículos y total). Esto permite que Inventario, Facturación y Envíos trabajen de forma autónoma, sin consultar al productor. El costo es un mensaje más grande y un mayor acoplamiento al esquema de datos.
- **Event Notification / evento ligero (thin):** PedidoCancelado transporta únicamente idPedido y motivo. Si un consumidor necesita más información, debe obtenerla por su cuenta. En la implementación, Inventario realiza un GET /pedidos/:idPedido cuando no tiene el pedido en su estado local. El costo es una dependencia temporal del productor; el beneficio es un mensaje pequeño y un menor acoplamiento estructural.

5.4 Topología del broker

RabbitMQ organiza la mensajería con tres elementos:

- **Exchange (intercambiador):** recibe los mensajes y decide a qué colas derivarlos. No almacena mensajes.

- **Binding (vinculación):** regla que conecta un exchange con una cola e indica qué routing keys acepta.
- **Queue (cola):** almacena los mensajes hasta que un consumidor los procesa. RabbitMQ (s. f., AMQP 0-9-1 Model Explained)

La topología implementada se resume en la Tabla 5.3.

Exchange	Tipo	Cola	Binding key	Servicio
intercambiador_pedidos	topic	cola_inventario	pedido.creado, pedido.cancelado	Inventario
intercambiador_pedidos	topic	cola_facturacion	pedido.creado, pedido.cancelado	Facturación
intercambiador_pedidos	topic	cola_envios	pedido.creado	Envíos
intercambiador_pedidos	topic	cola_notificaciones	# (todos los tópicos)	Notificaciones
intercambiador_dlq	fanout	cola_mensajes_muertos	—	Aislamiento de fallos

Tabla 5.3. Exchanges, colas y bindings.

El exchange topic permite enrutamiento selectivo mediante patrones. En un tópico, las palabras se separan por puntos y se admiten dos comodines: * reemplaza exactamente una palabra y # reemplaza cero o más palabras. En la implementación, cola_envios solo recibe pedido.creado, mientras que cola_notificaciones recibe todos los eventos porque está vinculada con #.

El exchange intercambiador_dlq es de tipo fanout: no evalúa routing keys y entrega el mensaje a todas las colas vinculadas. Se utiliza para derivar mensajes rechazados a la **Dead Letter Queue (DLQ)**, una cola especial donde se aíslan los mensajes que no pudieron procesarse.

Cada consumidor declara la topología al arrancar. Un ejemplo simplificado:

```
await canal.assertExchange('intercambiador_pedidos', 'topic', { durable: true });

await canal.assertQueue('cola_envios', {

  durable: true,

  arguments: { 'x-dead-letter-exchange': 'intercambiador_dlq' }

});

await canal.bindQueue('cola_envios', 'intercambiador_pedidos', 'pedido.creado');
```

Las colas y los exchanges son durable, y los mensajes se publican como persistent. Esto significa que sobreviven a un reinicio del broker. Además, los consumidores utilizan **ACK manual**: confirman el mensaje solo después de procesarlo correctamente; si falla, ejecutan NACK y el mensaje se deriva a la DLQ.

Aclaración: *ACK* (acknowledgement) es la confirmación de que un mensaje fue procesado. *NACK* es la confirmación negativa: el consumidor informa que no pudo procesarlo. El ACK manual evita perder mensajes si el consumidor se cae a mitad del procesamiento.

5.5 Microservicios

La Tabla 5.4 resume las responsabilidades de cada servicio.

Servicio	Consume	Responsabilidad	Emite	Estado local
api-pedidos	— (solo monitorea)	Recibe pedidos, publica eventos y reconstruye la proyección	PedidoCreado, PedidoCancelado	Pedidos, eventos y bitácora en memoria
servicio-inventario	pedido.creado, pedido.cancelado	Reserva o repone stock	StockReservado, StockRestituido	Stock, reservas, eventos procesados y versiones
servicio-facturacion	pedido.creado, pedido.cancelado	Emite factura con IVA 21 % o nota de crédito	FacturaEmitida, NotaCreditoEmitida	Facturas, eventos procesados y versiones
servicio-envios	pedido.creado	Genera guía de despacho y número de seguimiento	EnvioProgramado	Eventos procesados y versiones
servicio-notificaciones	# (todos)	Audita y correlaciona la cadena completa	No emite	Historial por idCorrelacion

Tabla 5.4. Responsabilidades de los microservicios.

Cada consumidor implementa tres mecanismos comunes:

1. **Idempotencia por idEvento:** mantiene un conjunto de eventos ya procesados. Si el mismo evento llega dos veces, lo ignora y confirma el mensaje sin repetir el efecto de negocio.
2. **Control de versiones:** compara la versión entrante con la última aplicada al pedido. Si es menor o igual, descarta el mensaje por obsoleto.
3. **DLQ:** si el procesamiento lanza una excepción, ejecuta NACK sin reencolar y el mensaje se deriva a cola_mensajes_muertos.

El servicio de inventario incorpora además el comportamiento para eventos ligeros: si recibe un PedidoCancelado y no tiene la reserva en memoria, consulta GET /pedidos/:idPedido para recuperar los artículos y reponer el stock. Esto muestra el costo del evento thin y permite compararlo con el evento ECST.

El servicio de notificaciones se suscribe con el comodín # y agrupa los eventos por idCorrelacion. Así reconstruye una cadena como:

PedidoCreado → StockReservado → FacturaEmitida → EnvioProgramado

→ PedidoCancelado → StockRestituido → NotaCreditoEmitida

5.6 Flujos principales

5.6.1 Creación de un pedido

1. El cliente envía POST /pedidos con idPedido, cliente, articulos y total.
2. api-pedidos valida los datos, guarda el pedido en memoria y publica PedidoCreado en pedido.creado.
3. La API responde 201 Created inmediatamente, sin esperar a los consumidores.
4. RabbitMQ copia el mensaje a las colas de Inventario, Facturación, Envíos y Notificaciones.
5. Cada consumidor procesa su parte y publica su propio evento de dominio.
6. La proyección del pedido en api-pedidos pasa de CREADO a EN_PROCESO y finalmente a COMPLETADO cuando se recibieron StockReservado, FacturaEmitida y EnvioProgramado.

Este flujo evidencia el **desacople temporal**: el productor no bloquea al cliente mientras los consumidores trabajan.

5.6.2 Cancelación de un pedido

1. El cliente envía POST /pedidos/:idPedido/cancelar.
2. api-pedidos publica PedidoCancelado en pedido.cancelado con solo idPedido y motivo.
3. Inventario, Facturación y Notificaciones reciben el evento. Envíos no lo recibe porque no tiene un binding para pedido.cancelado.
4. Inventario repone el stock; si no tenía la reserva en memoria, consulta la API para obtener los artículos.
5. Facturación emite una nota de crédito y anula la factura previa.
6. La proyección del pedido pasa a CANCELADO y luego a CANCELADO_Y_REPUESTO cuando llega StockRestituido.

Este flujo evidencia el **enrutamiento selectivo** y el uso de un evento ligero con callback.

5.6.3 Reintento duplicado e idempotencia

Se envía dos veces el mismo idEvento. El primer mensaje se procesa; el segundo se detecta como duplicado y se descarta sin volver a descontar stock ni emitir una segunda factura. El panel muestra la reacción como “Duplicado ignorado” o “Doble cobro evitado”.

5.6.4 Pedido inválido y DLQ

Se publica un pedido con simularFallo: true. Los consumidores lanzan una excepción controlada, ejecutan NACK y RabbitMQ deriva el mensaje a cola_mensajes_muertos. El resto de las colas sigue operando normalmente. Esto evidencia el **aislamiento de fallos**.

5.6.5 Caída y recuperación de un consumidor

Se detiene el contenedor de Facturación y se crea un pedido. La API responde igual, Inventario y Envíos procesan el evento, y la cola de Facturación retiene el mensaje. Al reiniciar el consumidor, este procesa el mensaje pendiente y la cola vuelve a cero. La proyección del pedido muestra EN_PROCESO con pendiente FacturaEmitida mientras Facturación está detenida, y pasa a COMPLETADO cuando se recupera.

Este flujo evidencia el **desacople temporal** y la **consistencia eventual**.

5.7 Panel de observación

La aplicación incluye un panel web servido por api-pedidos en http://localhost:3000. El panel consulta GET /estado-sistema cada 1,2 segundos y presenta cuatro pestañas:

1. **Simulador de eventos:** cuatro escenarios (crear, cancelar, duplicado y DLQ) y la reacción de cada microservicio.
2. **Colas y tópicos:** estado de las colas, consumidores activos, mensajes en espera, ACK y **bindings reales leídos del broker**.
3. **Sobres de eventos (JSON):** inspección del EventEnvelope completo de cada evento publicado.
4. **Pedidos y consistencia eventual:** proyección local del pedido con estado, versión, eventos recibidos y eventos pendientes.

El panel también muestra un indicador dinámico del estado de RabbitMQ: si el broker se detiene, el indicador cambia a "Desconectado"; al reconectarse, vuelve a "Conectado".

5.8 Decisiones de diseño y justificación

Decisión	Justificación	Costo o alternativa
RabbitMQ en lugar de Kafka	Ofrece enrutamiento flexible por tópicos, DLQ y una consola de administración; es adecuado para el volumen y la complejidad de la demo	Kafka sería preferible para alto throughput, retención prolongada y replay de eventos
Exchange topic	Permite enrutamiento selectivo por patrones y el comodín #	Requiere definir y mantener los bindings
Coreografía en lugar de orquestación	Los cuatro servicios reaccionan de forma independiente a un hecho común; no se necesita un coordinador central	En flujos con compensaciones complejas convendría una saga orquestada
ECST para creación y thin para cancelación	Permite mostrar los dos extremos de la taxonomía y comparar sus costos	ECST aumenta el tamaño del mensaje; thin introduce un callback
ACK manual y DLQ	Evita perder mensajes y aísla los errores sin frenar el flujo principal	Requiere manejar la DLQ y definir políticas de reintento
Docker Compose	Reproduce el entorno completo con un solo comando y evita el problema de "en mi máquina funciona"	En producción se usaría un orquestador de contenedores
Estado en memoria	Simplifica la demo y permite observar el comportamiento sin una base de datos	No persiste entre reinicios; en producción se usaría una base de datos o Redis

Tabla 5.5. Decisiones de diseño.

5.9 Limitaciones y mejoras

La implementación es una demo académica; sus limitaciones son conscientes y se documentan para evitar interpretaciones erróneas.

Limitación	Consecuencia	Mejora propuesta
Coreografía parcial	Facturación y Envíos no esperan a StockReservado; si el stock falla, podrían haber emitido su evento	Encadenar eventos o implementar una saga con compensaciones
Idempotencia en memoria	Al reiniciar un consumidor se pierden los eventos procesados y las versiones	Persistir en una base de datos o Redis
Sin publisher confirms	La API no espera confirmación del broker; un 201 no garantiza que el mensaje se haya persistido	Usar createConfirmChannel() y waitForConfirms()
Sin reintentos con backoff	Todo error va directamente a la DLQ, sin distinguir fallos transitorios de mensajes venenos	Cola de reintentos con TTL (tiempo de vida) y contador de intentos
Broker único	RabbitMQ es un punto único de falla	Clúster de RabbitMQ con réplicas y colas *quorum* (colas replicadas)
Sin schema registry	Los cambios en el sobre o en datos requieren coordinar a todos los servicios	Registrar y versionar esquemas
Sin persistencia de negocio	Pedidos, stock y facturas se pierden al reiniciar	Base de datos transaccional
Sin autenticación	Los endpoints y el panel no están protegidos	Autenticación, autorización y secretos por entorno
Sin pruebas automatizadas	La verificación es manual mediante la demo	Pruebas unitarias y de integración

Tabla 5.6. Limitaciones y mejoras.

5.10 Evidencias de funcionamiento

Puesta en marcha

Repositorio: https://github.com/Malenatrangoni/ecommerce-eda-demo.git

docker compose up --build -d

El comando construye las imágenes y levanta seis contenedores: RabbitMQ y los cinco microservicios. La API y el panel quedan disponibles en http://localhost:3000; la consola de RabbitMQ, en http://localhost:15672 (usuario guest, contraseña guest).

Caso 1. Creación de un pedido: el evento como hecho y la coreografía

Qué se hace. En la pestaña 1 se pulsa “1. Crear Pedido de Compra”, que equivale a POST /pedidos. La API responde 201 de inmediato, sin esperar a que los consumidores terminen.

Qué se observa. api-pedidos publica PedidoCreado en pedido.creado con el estado completo del pedido. RabbitMQ copia el evento a las cuatro colas. Inventario descuenta stock y publica StockReservado; Facturación emite la factura y publica FacturaEmitida; Envíos genera la guía y publica EnvioProgramado; Notificaciones audita la cadena. En la pestaña 1, las cuatro tarjetas quedan verdes y muestran el evento que emitió cada servicio.

Qué concepto demuestra. EDA (el evento es un hecho inmutable), ECST o evento fat (PedidoCreado lleva cliente, artículos y total), coreografía (cada consumidor reacciona y emite su propio evento) y desacople espacial (el productor no conoce a los consumidores).

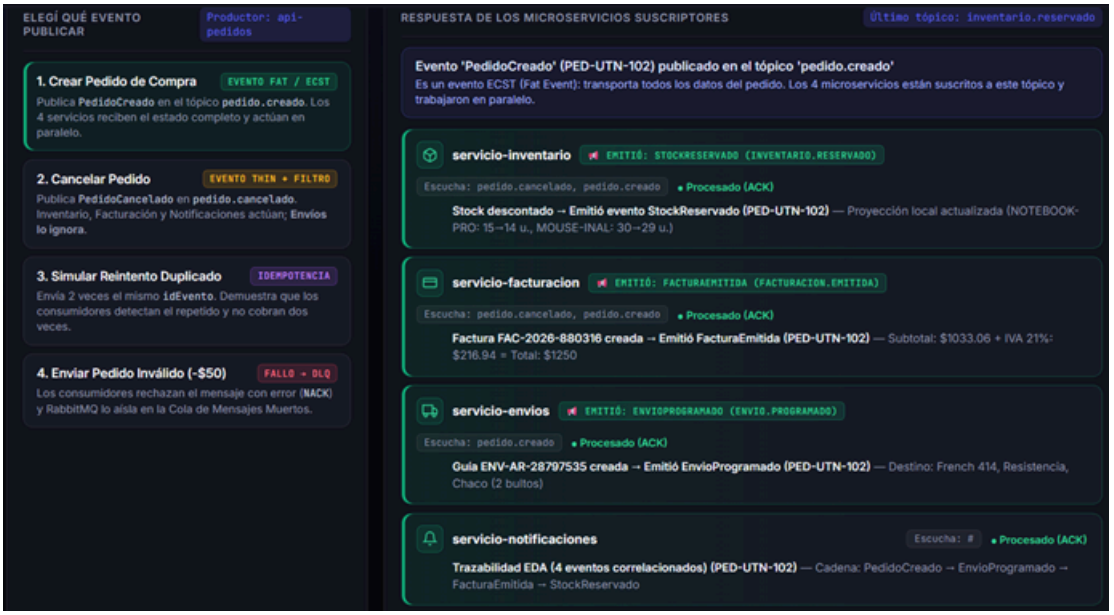


Figura 5.2. Creación de un pedido: los cuatro consumidores reaccionan en paralelo y emiten sus propios eventos.

Caso 2. Cancelación: evento ligero y enrutamiento selectivo

Qué se hace. Se pulsa “2. Cancelar Pedido” sobre el pedido recién creado. La API publica PedidoCancelado en pedido.cancelado.

Qué se observa. El evento solo lleva idPedido y motivo. RabbitMQ lo entrega a Inventario, Facturación y Notificaciones; Envíos no lo recibe porque su binding solo acepta pedido.creado. Inventario repone el stock y, si no tiene la reserva en caché, consulta GET /pedidos/:id (callback). Facturación emite la nota de crédito. En la pestaña 1, la tarjeta de Envíos queda “Filtrado por Tópico” y las otras tres en ámbar.

Qué concepto demuestra. Event Notification o evento thin (el evento no lleva el estado completo), exchange topic y binding selectivo, comodín # en Notificaciones, costo del callback y consistencia eventual.

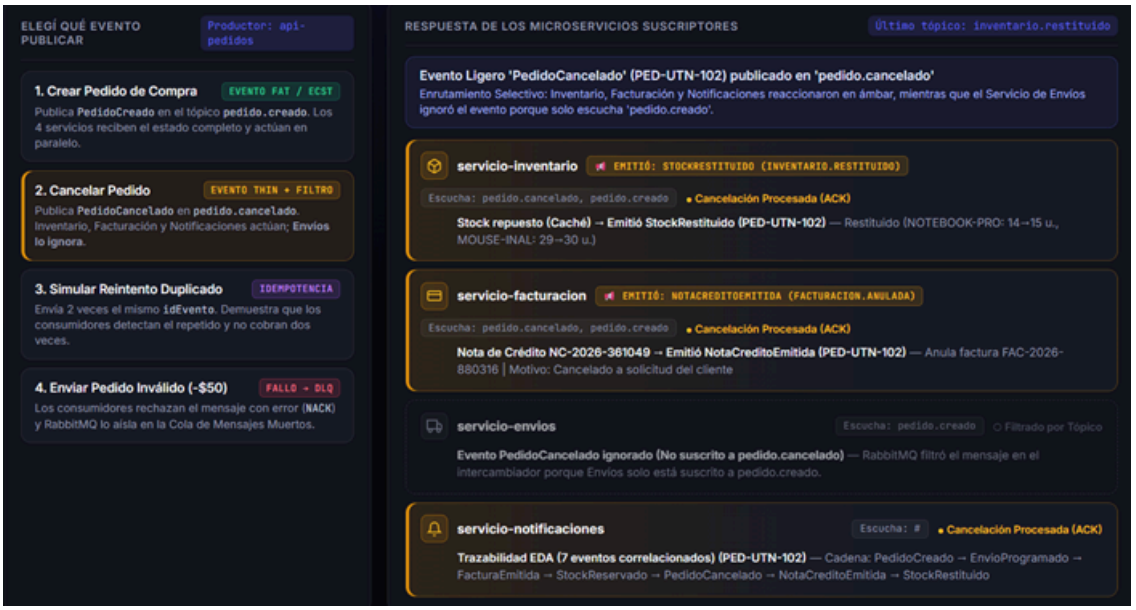


Figura 5.3. Cancelación: Envíos no recibe el evento por el filtro de tópico.

Caso 3. Reintento duplicado: la idempotencia

Qué se hace. Se pulsa "3. Simular Reintento Duplicado". La demo envía dos veces el mismo idEvento, simulando un reintento de red.

Qué se observa. El primer mensaje se procesa. El segundo se detecta como duplicado por el conjunto de eventos procesados y por el control de versiones, y se descarta con ACK. No se vuelve a descontar stock ni se emite una segunda factura. Las tarjetas se muestran en violeta con "Duplicado ignorado" o "Doble cobro evitado".

Qué concepto demuestra. Garantía at-least-once (los duplicados son posibles), idempotencia (el efecto se aplica una sola vez) y control de versiones para descartar mensajes obsoletos.

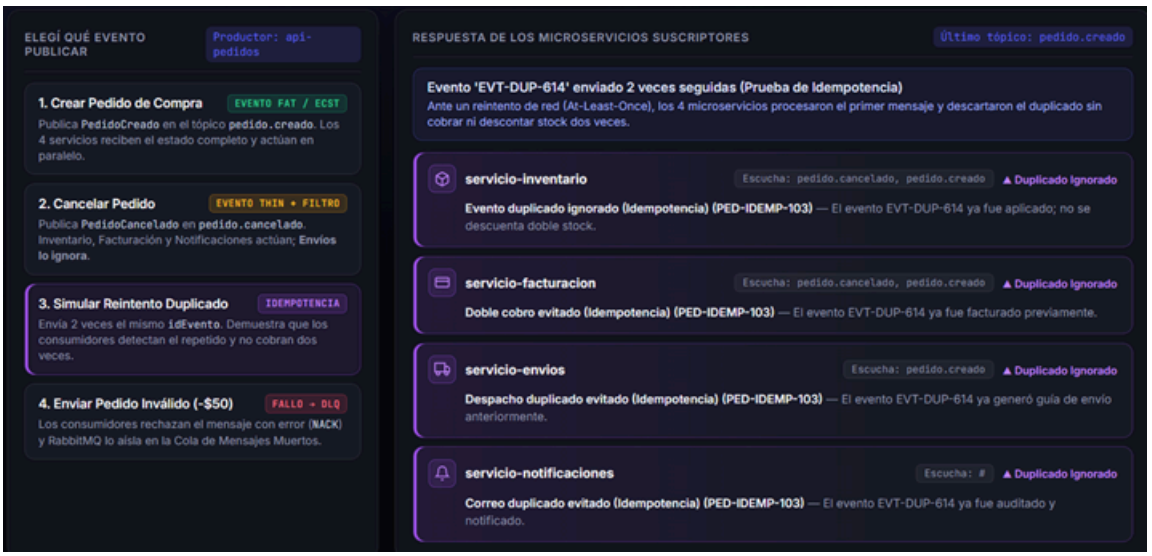


Figura 5.4. Reintento duplicado: los consumidores descartan el segundo evento sin repetir el efecto de negocio.

Caso 4. Pedido inválido: NACK y aislamiento en la DLQ

Qué se hace. Se pulsa “4. Enviar Pedido Inválido”. El pedido se publica con simularFallo y un monto negativo.

Qué se observa. Inventario, Facturación y Envíos rechazan el mensaje con NACK sin reencolar; RabbitMQ lo deriva a cola_mensajes_muertos mediante el dead-letter exchange. Notificaciones solo audita y no rechaza. El resto de las colas sigue operando. En la pestaña 2, la fila de la DLQ aparece con mensajes.

Qué concepto demuestra. ACK/NACK manual, Dead Letter Queue, aislamiento de fallos y diferencia entre un error de negocio y un mensaje venenoso.

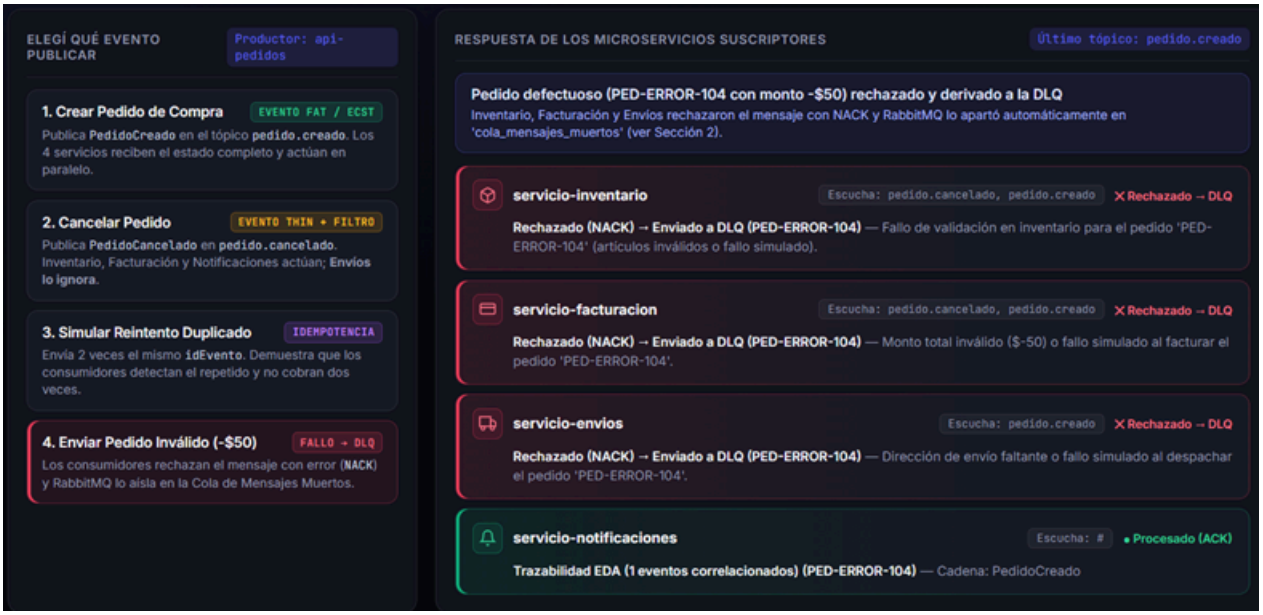


Figura 5.5. Pedido inválido: el mensaje rechazado se aísla en la DLQ sin frenar el resto del sistema.

Caso 5. Facturación caída: desacople temporal y consistencia eventual

Qué se hace. Se detiene Facturación con docker compose stop servicio-facturacion y se crea un pedido nuevo.

Qué se observa. La API responde 201 igual. Inventario, Envíos y Notificaciones procesan el evento; la cola de Facturación retiene el mensaje (1 mensaje listo, 0 consumidores). La proyección del pedido queda EN_PROCESO con pendiente FacturaEmitida: el pedido existe y la venta sigue, pero el estado todavía no está completo.

Qué concepto demuestra. Desacople temporal (productor y consumidor no necesitan estar disponibles al mismo tiempo), colas durables y mensajes persistentes, y consistencia eventual (el sistema converge a COMPLETADO cuando Facturación vuelve).



Figura 5.6. Facturación detenida: el pedido queda EN_PROCESO y el mensaje espera en la cola.

Caso 6. Facturación vuelve: recuperación y convergencia

Qué se hace. Se reinicia Facturación con docker compose start servicio-facturacion y se esperan unos segundos.

Qué se observa. El consumidor reanuda la conexión, lee el mensaje pendiente, emite FacturaEmitida y la cola vuelve a 0. La proyección del pedido pasa a COMPLETADO.

Qué concepto demuestra. Desacople temporal (el mensaje no se perdió durante la caída), at-least-once (el mensaje se entrega cuando el consumidor vuelve) y convergencia de la consistencia eventual.



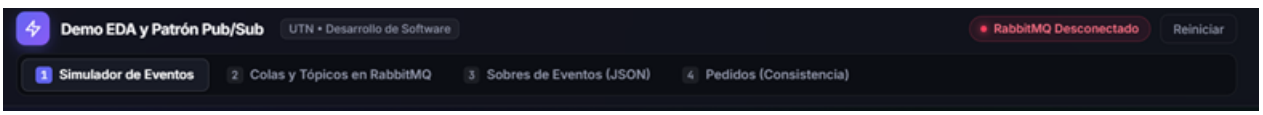
Figura 5.7. Al reiniciar Facturación, el mensaje pendiente se procesa y el pedido se completa.

Caso 7. Broker caído: el límite de la arquitectura

Qué se hace. Se detiene RabbitMQ con docker compose stop rabbitmq y luego se vuelve a iniciar.

Qué se observa. El indicador del panel pasa a “RabbitMQ Desconectado” en rojo y, al reconectar, vuelve a “Conectado”. Los servicios reintentan la conexión automáticamente.

Qué concepto demuestra. El broker es un componente crítico y un punto único de falla; la observabilidad permite detectar la caída; en producción se usaría un clúster con réplicas.



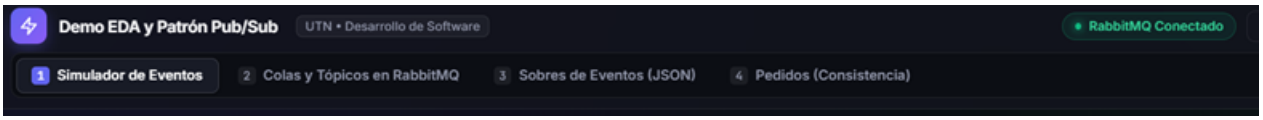


Figura 5.8 El panel refleja la caída y la reconexión del broker.

La secuencia muestra cómo cada concepto de la investigación se materializa en una decisión concreta de la implementación: el evento completo en la creación, el evento ligero en la cancelación, el binding selectivo, la idempotencia ante duplicados, la DLQ ante errores, el desacople temporal cuando un consumidor se cae y la convergencia de la consistencia eventual cuando vuelve.

5.11 Cierre de la sección

La implementación demuestra que EDA y Pub/Sub pueden integrarse en una solución concreta sin recurrir a llamadas síncronas en cadena. El productor publica hechos, el broker los distribuye según reglas explícitas y cada consumidor reacciona de forma autónoma, con idempotencia, versionado y aislamiento de fallos. La demo permite observar tanto las ventajas (desacople, tolerancia a fallos y extensibilidad) como los costos (consistencia eventual, complejidad operativa y necesidad de observabilidad).

6. Conclusiones

Este trabajo investigó la Arquitectura Orientada a Eventos (EDA) y el patrón de Publicación/Suscripción (Pub/Sub) como respuesta al acoplamiento que generan las llamadas síncronas directas entre servicios. La investigación mostró que ambos conceptos se complementan: EDA define qué se comunica, es decir, hechos del dominio que ya ocurrieron y se expresan como eventos, y Pub/Sub define cómo se distribuyen, de modo que el productor publica una sola vez y cada consumidor reacciona de forma independiente, sin que el productor lo conozca.

Aplicados a la plataforma de movilidad urbana del TPI, estos conceptos encajan de manera natural en el módulo M6: cada transición de estado de un viaje (TripCreated, DriverArrived, TripStarted, TripCompleted y TripCancelled) es un hecho que interesa a varios módulos a la vez, como para revisar el historial, calcular costos, etc. La distinción entre eventos thin y fat permitió decidir qué datos debe llevar cada evento: señales livianas para avisos inmediatos y eventos completos cuando el consumidor necesita actuar sin consultar a M6. Del mismo modo, la coreografía resulta adecuada entre módulos independientes, mientras que la orquestación no hace falta en M6 y se reserva para flujos con pasos condicionales y compensaciones.

La implementación práctica, un e-commerce simplificado con RabbitMQ y cinco microservicios, permitió comprobar estas ideas con evidencia. Se observó el desacople espacial (la API responde sin conocer a los consumidores), el enrutamiento selectivo (Envíos no recibe las cancelaciones), la idempotencia frente a mensajes duplicados, el aislamiento de fallos mediante la Dead Letter Queue y el desacople temporal: cuando Facturación se detuvo, el mensaje esperó en la cola y el pedido se completó al volver el servicio. La demo también

dejó ver los costos de la arquitectura: consistencia eventual, el callback que exige el evento thin, la dependencia de un broker que es punto único de falla y la mayor complejidad de operación y de depuración.

En consecuencia, EDA y Pub/Sub convienen cuando un mismo hecho interesa a varios dominios independientes y se necesita tolerar caídas y absorber picos de carga. No convienen para consultas que requieren una respuesta inmediata y consistente (como obtener el estado de un viaje), ni para validaciones bloqueantes (como verificar el token QR), ni en sistemas simples con un único consumidor, donde el broker agrega costo sin aportar beneficio.

7. Bibliografía

Apache Software Foundation. (s. f.). *Apache Kafka documentation*. https://kafka.apache.org/documentation/ (Consultado el 09/11/2026).

Eugster, P. T., Felber, P. A., Guerraoui, R., & Kermarrec, A.-M. (2003). The many faces of publish/subscribe. *ACM Computing Surveys, 35*(2), 114-131. https://infoscience.epfl.ch/record/52445

Fowler, M. (2017). *What do you mean by "Event-Driven"?* https://martinfowler.com/articles/201701-event-driven.html (Consultado el 02/11/2026).

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Hohpe, G., & Woolf, B. (s. f.). *Competing consumers*. Enterprise Integration Patterns. https://www.enterpriseintegrationpatterns.com/patterns/messaging/CompetingConsumers.html (Consultado el 02/11/2026).

Hohpe, G., & Woolf, B. (s. f.). *Dead letter channel*. Enterprise Integration Patterns. https://www.enterpriseintegrationpatterns.com/patterns/messaging/DeadLetterChannel.html (Consultado el 02/11/2026).

Hohpe, G., & Woolf, B. (s. f.). *Publish-subscribe channel*. Enterprise Integration Patterns. https://www.enterpriseintegrationpatterns.com/patterns/messaging/PublishSubscribeChannel.html (Consultado el 02/11/2026).

Microsoft. (s. f.). *Estilo de arquitectura controlada por eventos*. Azure Architecture Center. https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/event-driven (Consultado el 02/11/2026).

Microsoft. (s. f.). *Patrón Publicador-Suscriptor*. Azure Architecture Center. https://learn.microsoft.com/es-es/azure/architecture/patterns/publisher-subscriber (Consultado el 02/11/2026).

RabbitMQ. (s. f.). *AMQP 0-9-1 model explained*. https://www.rabbitmq.com/tutorials/amqp-concepts (Consultado el 06/11/2026).

RabbitMQ. (s. f.). *Dead letter exchanges*. https://www.rabbitmq.com/docs/dlx (Consultado el 05/11/2026).

Richardson, C. (s. f.). *Pattern: Saga*. Microservices.io. https://microservices.io/patterns/data/saga.html (Consultado el 02/11/2026).

Richardson, C. (s. f.). *Pattern: Transactional outbox*. Microservices.io. https://microservices.io/patterns/data/transactional-outbox.html (Consultado el 02/11/2026).

Repositorio de la demo ecommerce:
https://github.com/Malenatrangoni/ecommerce-eda-demo.git